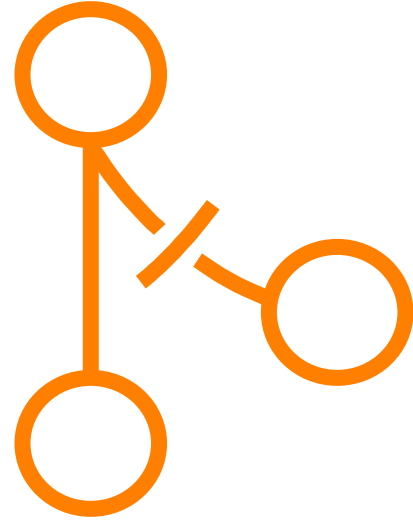


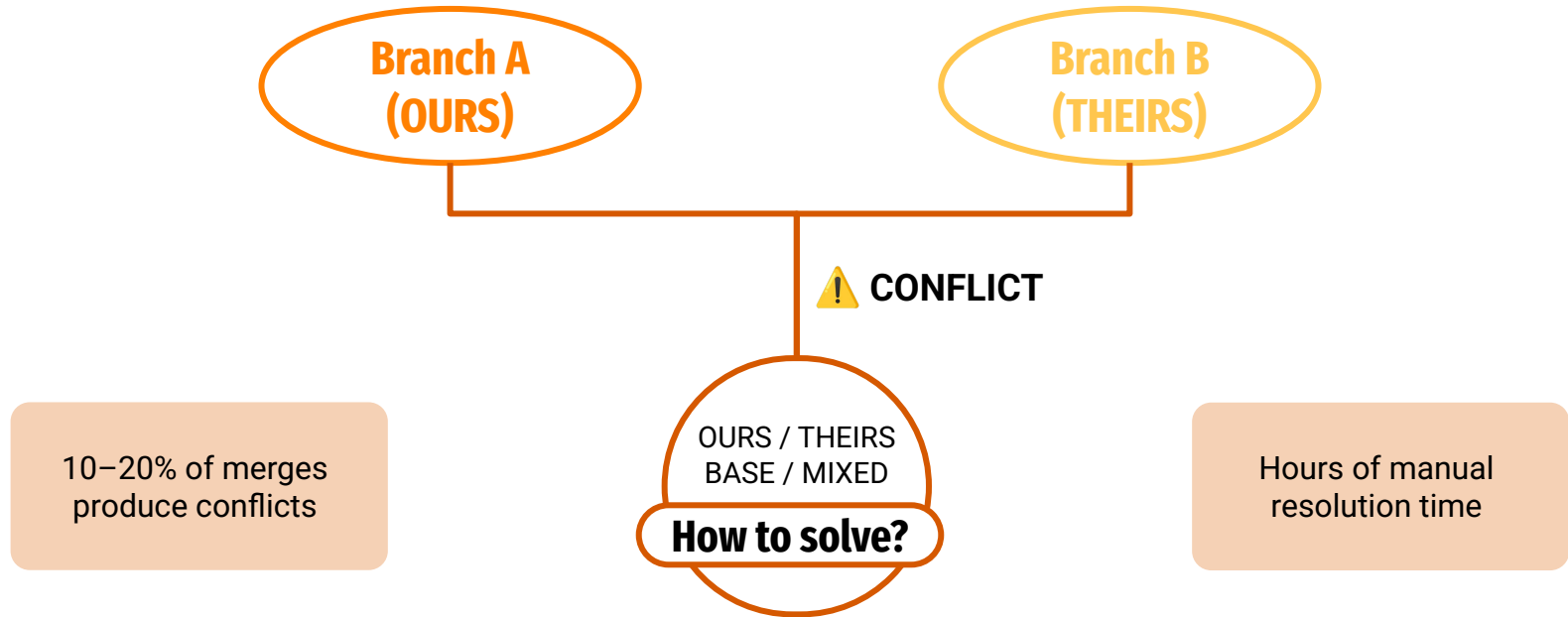
Predicting Merge Conflict Resolutions: WSRC vs. Random Forest

Asier Yániz Ibáñez

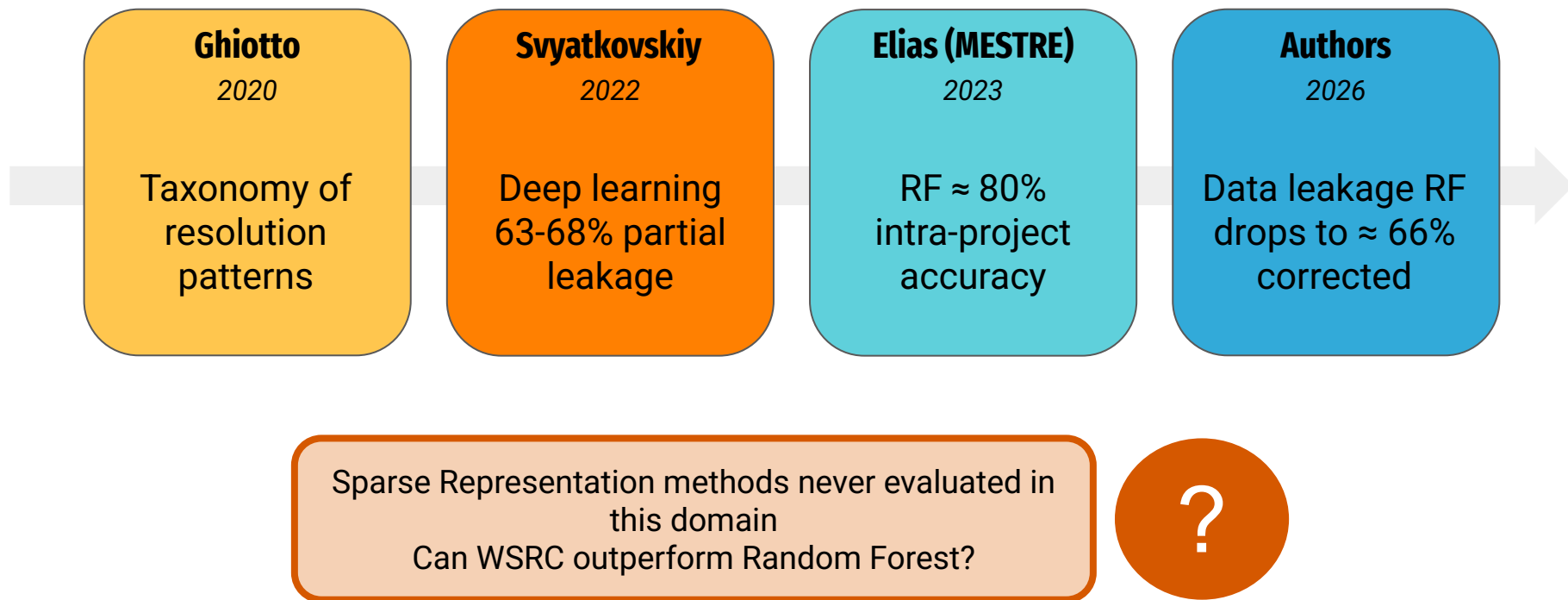
Seminar Software Engineering 2026



Motivation and Context



Previous Work and Research Gap



Dataset and Features



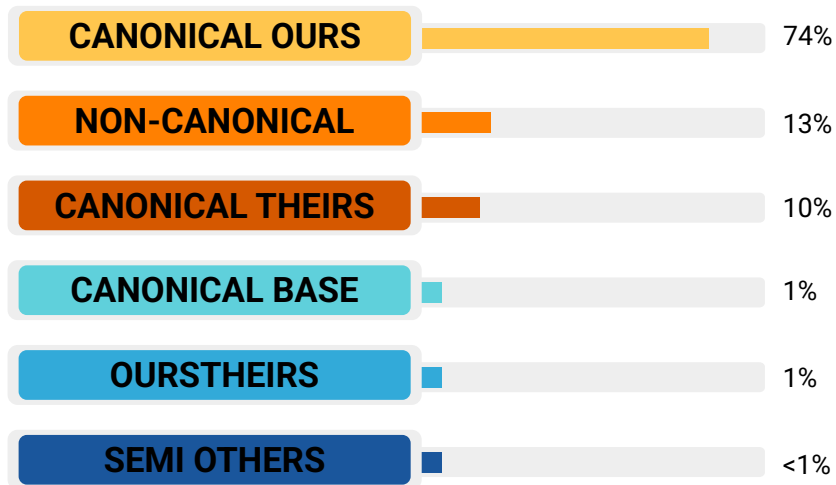
16 projects open-source Java



9,698 merges commits



140,127 chunks



 Authorship
 Commit msg

 Change size
 Complexity

 Temporal
 Chunk location

Evaluation Protocol: the Data Leakage Problem

✗ S1 - Chunk-level split

Merge A

Chunk 1 - TRAIN

Chunk 2 - TEST

Chunk 3 - TRAIN

Merge B

Chunk 4 - TEST

Chunk 5 - TRAIN

Chunk 6 - TEST

✓ S3 - Merge-level grouping

Merge A

Chunk 1 - TRAIN

Chunk 2 - TRAIN

Chunk 3 - TRAIN

Merge B

Chunk 4 - TEST

Chunk 5 - TEST

Chunk 6 - TEST



S1 leakage ratio across projects: avg. 96%

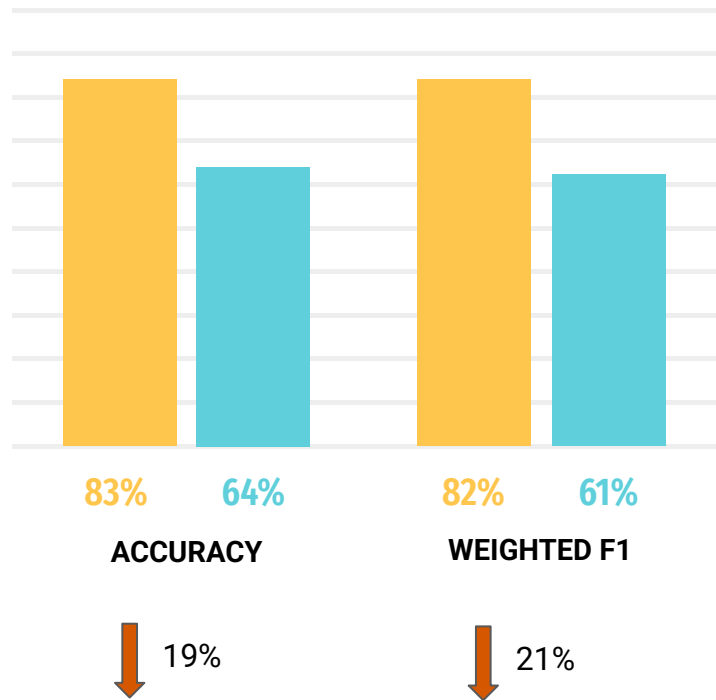
Replicating the Data Leakage Experiment

S1

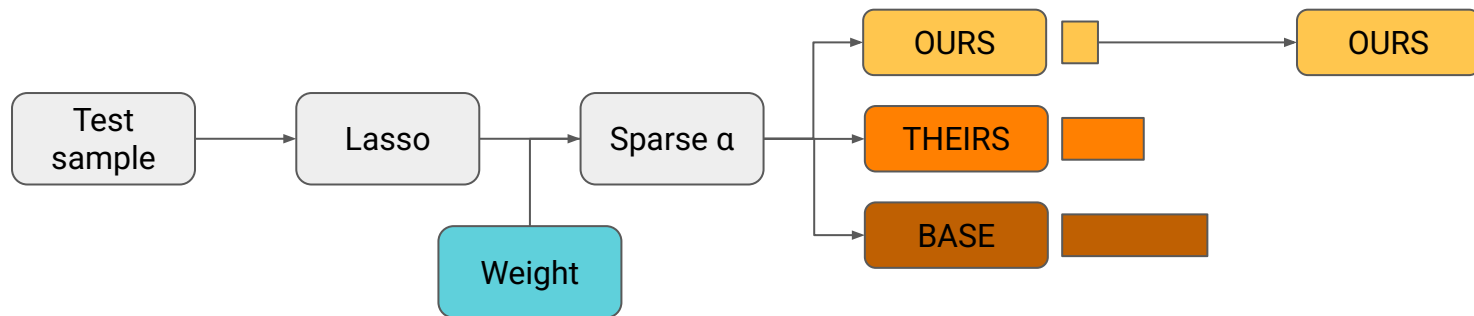
Results matches
Elias ones (2023)
 $0.83 = 0.83$

S3

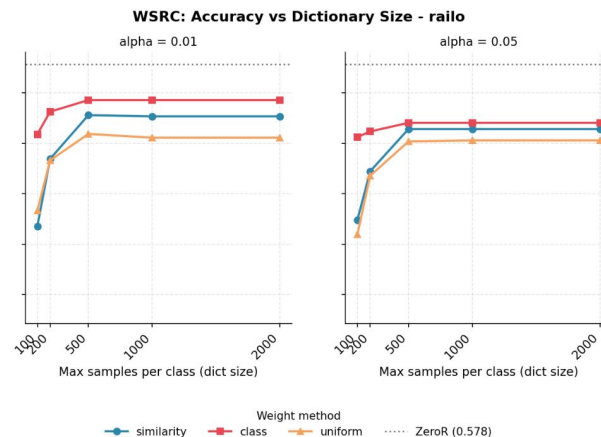
Results matches
Authors ones (2026)
 $0.66 \approx 0.64$



How SRC and WSRC Work



PARAMETER	ACCURACY
α (Lasso)	0.001 / 0.01 / 0.05 / 0.1
Dict size	100 / 200 / 500 / 1000 / 2000
Weight	similarity / class / uniform



Compared Models

Random Forest

400 trees
Default sklearn settings
No normalisation needed
Baseline

KNN

Euclidean distance
Inverse distance weighting
K via grid search
Proximity baseline

SRC

Sparse linear reconstruction
Lasso $\alpha = 0.01$ and ≤ 500 atoms/class
StandardScaler normalisation

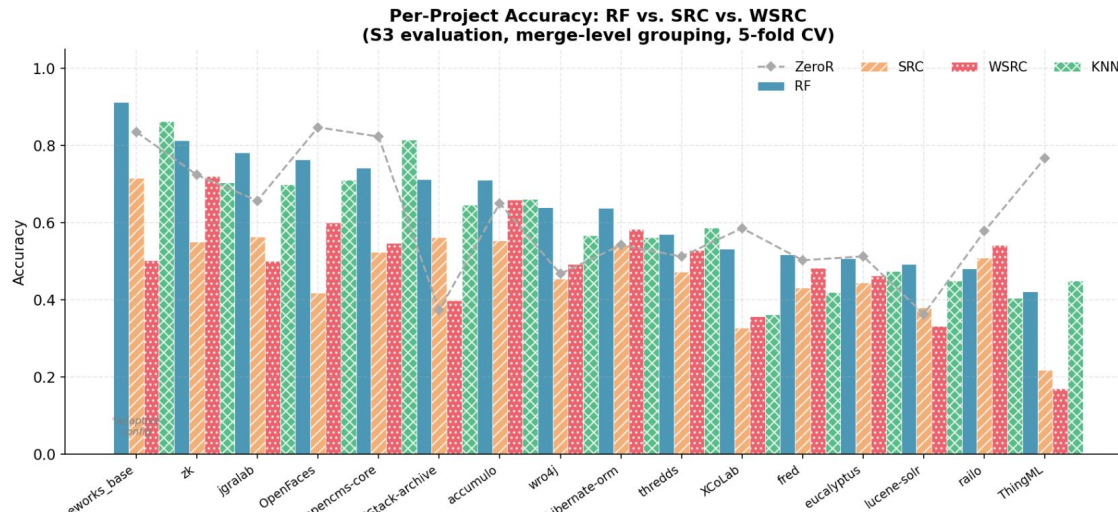
WSRC

SRC + class-frequency weighting
StandardScaler normalisation
Challenger

SRC and WSRC share the exact same dictionary -> isolates the effect of weighting

Main Results

Random Forest
leads
15/16 projects



aosp-mirror (4,500 merges) -> **91%**

ThingML (52 merges) -> **43%**

Friedman $\chi^2(3) = 26.47$

$p < 0.001$

→ Models are **not equivalent**

RF vs. Rest

Wilcoxon test

RF vs. WSRC

$P = 0.0004$
 $\Delta = +0.147$

RF vs. SRC

$P = 0.0001$
 $\Delta = +0.160$

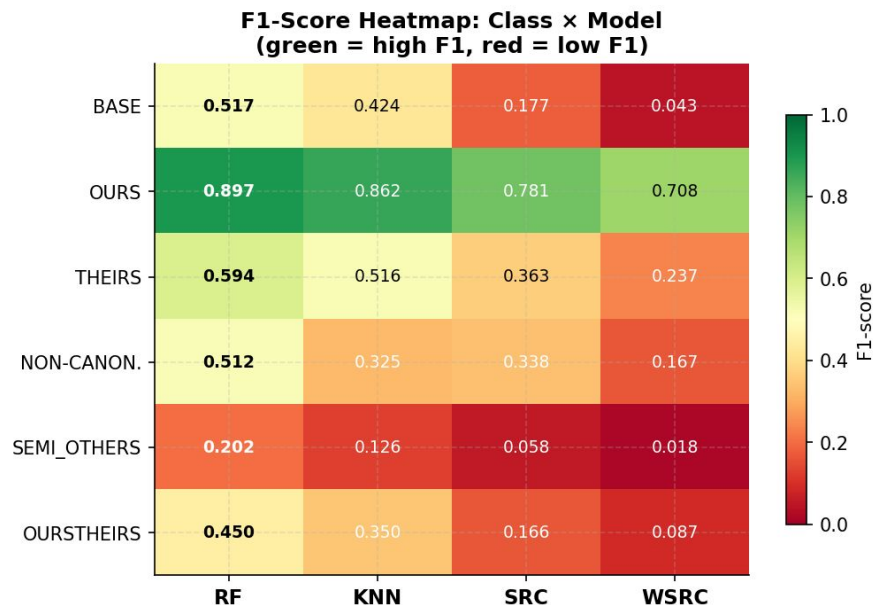
RF vs. KNN

$P = 0.0021$
 $\Delta = +0.054$

MODEL	ACCURACY	WINS
RF	0.640	15/16
KNN	0.591	3/16
WSRC	0.490	1/16
SRC	0.473	0/16
ZeroR	0.609	

RF is significantly better than all competitors
But: is WSRC at least better than plain SRC?

Per-Class Analysis



F1

F1 degrades monotonically with class rarity across ALL models

WSRC

WSRC collapses on minority classes:

BASE: 0.044 (RF: 0.517)

SEMI OTHERS: 0.018 (RF: 0.202)

SRC vs. WSRC (Wilcoxon)

$p = 0.528$ - not significant

Weighting adds no measurable benefit

Is Class Imbalance to Blame?

Undersampling to equal classes

Δ accuracy ≈ -0.26 for BOTH
Ranking unchanged

RF

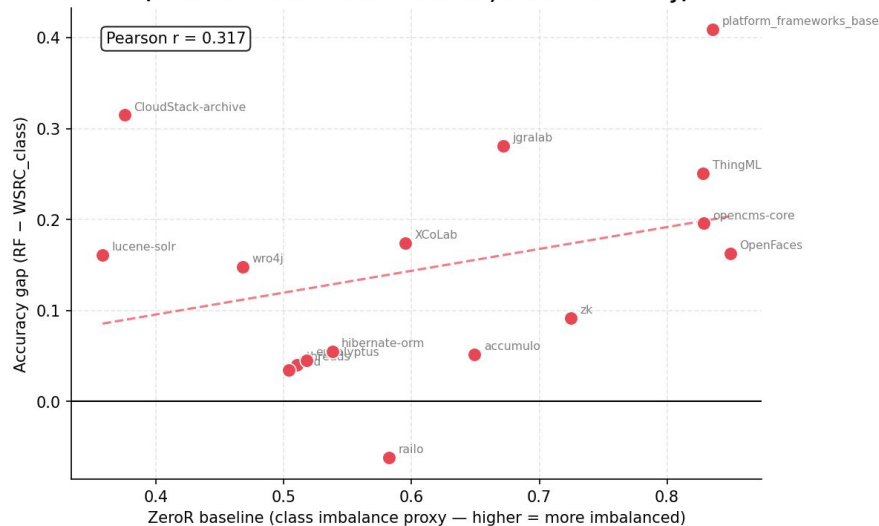
class_weight='balanced'
 $\Delta = +0.003 \rightarrow$ No effect

Spearman $\rho = 0.38$

$\rho = 0.15$ - not significant

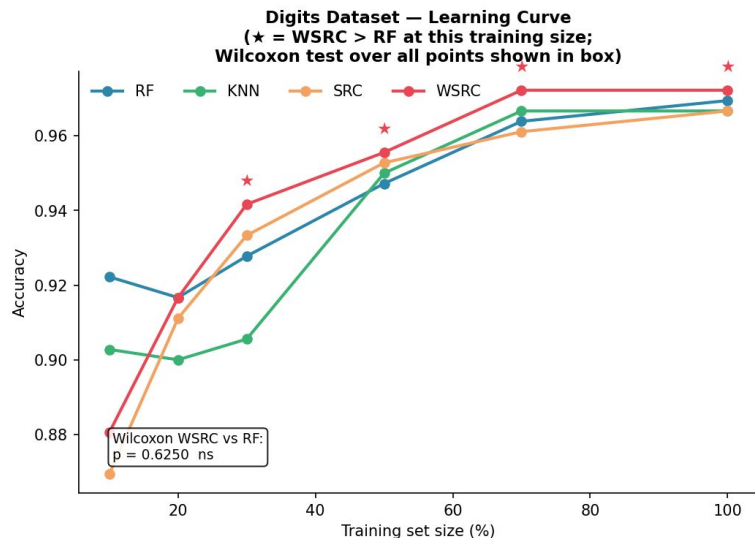
Imbalance does not explain the RF-WSRC gap

Does Class Imbalance Explain the RF-WSRC Gap?
($r=0.317$ — moderate correlation, not the full story)



WSRC's disadvantage is structural, not
distributional

Benchmark Experiments: Does WSRC Work Under Favorable Conditions?



Wilcoxon WSRC vs. SRC
On Digits: $p = 0.031$
Weighting DOES work here

MODEL	WINE (ACC)	DIGITS (ACC)
RF	0.983	0.980
KNN	0.983	0.981
WSRC	0.955	0.972
SRC	0.860	0.967
ZeroR	0.399	0.102

- ✓ Balanced classes
- ✓ High-dimensional features
- ✓ Linear subspace assumption holds
- ✗ None of these apply to merge conflicts

Discussion: Why Does WSRC Fail?

Feature Heterogeneity

43 features mixing
Euclidean geometry $\neq$ class geometry

Dictionary Domination

74% OURS exhausts dictionary capacity
regardless of weighting
Minority classes $\rightarrow$ too few discriminative
atoms



The bottleneck is not the classifier, it's the feature space

Conclusions

01

RF significantly
outperforms WSRC

02

Weighting adds no
benefit WSRC = SRC

03

The bottleneck is the
feature space

04

WSRC works when
conditions are right

05

Classical ML may be
approaching its
performance ceiling
for merge conflict
prediction

Future Work

The bottleneck is the feature space, not the classifier

Richer Representations

Current: 43 numerical metadata features around the conflict

Next: LLMs reading raw code changes -> capture developer intent directly

Minority Class Handling

Current: class imbalance hurts all models equally

Next: generative oversampling, cost-sensitive learning

Broader Evaluation

Current: 16 Java projects

Next: other languages, ecosystems, cross-project generalisation

Thanks for your attention Questions?

**Predicting Merge Conflict Resolutions:
WSRC vs. Random Forest**

Asier Yániz Ibáñez

Seminar Software Engineering 2026

